
`github.com/mg-i-roc-maz/corsi · microservizi/pre-corso`

MODULO PRE-CORSO · 2 ORE · FACOLTATIVO

Terminale, HTTP e JSON

Le tre basi che servono dal primo minuto del corso Microservizi, Caching ed Event-Driven.

Rocco Mazzeo

`rocco.mazzeo@gmail.com`

Fabio Rillo

`fabiorillo.work@gmail.com`

Le due ore

Terminale · 45'

Terminale

Muoversi tra le cartelle, creare file, lanciare comandi.

HTTP · 40'

HTTP

Come parlano client e server.
Prove con curl.

JSON · 25'

JSON

Il formato dei dati che viaggiano.

Git · 10'

Clonare il repo e la checklist per la lezione 1.

Che cos'è il terminale

Un modo di usare il computer scrivendo comandi invece di cliccare. Il programma che li interpreta si chiama **shell**.

Mac

App Terminale, shell **zsh**

Linux

Terminale, di solito **bash**

Windows

PowerShell, o Git Bash

VS Code

Terminale integrato: Ctrl+`
o Cmd+`

```
studente@pc ~ % pwd  
/Users/studente
```

Dove sono? Cartelle e percorsi

```
/
└─ Users
    └─ studente ← ~
        └─ Documenti
            └─ corsi
                └─ microservizi
```

Simbolo

Significato

/

La radice di tutto (Mac, Linux)

~

La tua cartella home

.

La cartella in cui sei

..

La cartella sopra

Assoluto: parte da / → /Users/studente/corsi

Relativo: parte da dove sei → corsi/microservizi

I comandi che userai ogni giorno

Comando	Che cosa fa	Esempio
pwd	Mostra dove sei	pwd
ls	Elenca i file della cartella	ls -la
cd	Entra in una cartella	cd corsi
mkdir	Crea una cartella	mkdir prove
touch	Crea un file vuoto	touch note.txt
cat	Mostra il contenuto di un file	cat note.txt
cp • mv	Copia • sposta o rinomina	mv a.txt b.txt
rm	Cancella, senza cestino	rm b.txt

Anatomia di un comando

npm

il **comando**: il programma da lanciare

install

un **argomento**: su cosa lavorare

--save-dev

un'**opzione**: come farlo

tsx

un altro **argomento**

Opzioni corte: un trattino e una lettera, **ls -l**. Si possono unire: **ls -la**.

Opzioni lunghe: due trattini e una parola, **--help**. Quasi ogni comando lo accetta.

Cinque tasti che fanno risparmiare ore

Tab

Completa nomi di file e cartelle



Ripercorre i comandi già scritti

Ctrl+C

Ferma il programma in corso, per esempio un server

clear

Pulisce lo schermo (anche Ctrl+L)

code .

Apri la cartella corrente in VS Code



Errore? Leggi l'ultima riga: di solito dice cosa manca.

Costruisci il tuo negozio

1. Nella home crea la cartella **negozio** ed entraci
2. Dentro crea **catalogo** e **ordini**
3. In catalogo crea **prodotti.txt**
4. Torna in negozio ed elenca tutto con `ls`
5. Rinomina ordini in **vendite**
6. Cancella `prodotti.txt`

RISULTATO ATTESO

```
~/negozio  
├─ catalogo  (vuota)  
└─ vendite
```

Ogni volta che sei in dubbio: **pwd** e **ls**.

Client e server



Il client chiede, il server risponde. **HTTP** è la lingua comune: regole su come scrivere richieste e risposte.

Anatomia di un URL

URL · *Uniform Resource Locator*: dice **dove** si trova una risorsa e come raggiungerla.

URI · *Uniform Resource Identifier*: la **identifica**. Ogni URL è anche un URI.

`http://localhost:3001/products/p-1?q=mouse`

Parte	Nell'esempio	A cosa serve
Schema	http	Il protocollo; https se cifrato
Host	localhost	Il computer: localhost è il tuo
Porta	3001	Quale programma su quel computer
Percorso	/products/p-1	Quale risorsa
Query	?q=mouse	Parametri extra: filtri, ricerche

Una richiesta e la sua risposta

RICHIESTA

```
GET /todos/1 HTTP/1.1
Host: jsonplaceholder.typicode.com
Accept: application/json
```

RISPOSTA

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "userId": 1, "id": 1,
  "title": "delectus aut autem",
  "completed": false }
```

Prima riga: metodo e percorso, o il codice di stato

Header: informazioni sul messaggio

Corpo: i dati, dopo una riga vuota

I metodi: il verbo della richiesta

Metodo	Significato	Esempio
GET	Leggi	GET /products
POST	Crea qualcosa di nuovo	POST /orders
PUT	Sostituisci tutto	PUT /products/p-1
PATCH	Modifica una parte	PATCH /products/p-1
DELETE	Cancella	DELETE /products/p-1

Quando scrivi un indirizzo nel browser fai sempre una **GET**. Per gli altri metodi servono strumenti come curl o REST Client.

I codici di stato: com'è andata

2xx

Tutto bene

200 OK
201 Created
204 No Content

3xx

Vai altrove

301 Moved
304 Not Modified

4xx

Errore del client

400 Bad Request
401 Unauthorized
404 Not Found

5xx

Errore del server

500 Internal
503 Unavailable

Basta la prima cifra per capire di chi è il problema: **4** hai sbagliato tu, **5** ha sbagliato il server.

Gli header più comuni

Header	Dove	A cosa serve
Content-Type	entrambi	Di che tipo è il corpo: application/json
Accept	richiesta	Che tipo di risposta voglio
Authorization	richiesta	Chi sono: un token o una password
Location	risposta	Dove trovare la risorsa appena creata
Cache-Control	risposta	Per quanto si può riusare la risposta

curl: HTTP dal terminale

Una GET

```
curl https://jsonplaceholder.typicode.com/todos/1
```

Con gli header della risposta

```
curl -i https://jsonplaceholder.typicode.com/todos/1
```

Una POST con un corpo JSON

```
curl -X POST https://jsonplaceholder.typicode.com/posts \  
-H "Content-Type: application/json" \  
-d '{"title":"ciao","userId":1}'
```

-i mostra gli header · **-X** sceglie il metodo · **-H** aggiunge un header · **-d** invia un corpo. Su Windows scrivi **curl.exe**.

Interroga un'API vera

Con curl, su <https://jsonplaceholder.typicode.com>. Per ognuna annota metodo, codice di stato e una cosa che leggi nella risposta.

#	Richiesta	Domanda
1	GET /users/1	Come si chiama l'utente?
2	GET /posts?userId=1	Quanti post ha scritto?
3	GET /posts/9999	Che codice ottieni?
4	POST /posts con un tuo titolo	Codice e id assegnato?
5	curl -i GET /todos/1	Che Content-Type?

Che cos'è JSON

JavaScript Object Notation: un formato di testo per rappresentare dati. Lo leggono le persone e tutti i linguaggi di programmazione.

È il formato con cui i nostri servizi si scambiano prodotti, ordini ed eventi.

```
{  
  "id": "p-1",  
  "name": "Tastiera",  
  "priceCents": 8990,  
  "available": true,  
  "tags": ["pc", "gaming"]  
}
```

Un **oggetto** sta tra graffe { } ed è fatto di coppie **chiave: valore**, separate da virgole.

I sei tipi di valore

Tipo	Esempio	Nota
Stringa	"Tastiera"	Sempre virgolette doppie
Numero	8990 · 3.5 · -2	Senza virgolette, punto decimale
Booleano	true · false	Minuscolo, senza virgolette
Null	null	«Nessun valore»
Array	[1, 2, 3]	Elenco ordinato, tra quadre
Oggetto	{ "a": 1 }	Coppie chiave: valore, tra graffe

Gli errori più comuni

Errore	Sbagliato	Giusto
Virgolette singole	<code>{ 'name': 'a' }</code>	<code>{ "name": "a" }</code>
Chiave senza virgolette	<code>{ name: "a" }</code>	<code>{ "name": "a" }</code>
Virgola finale	<code>{ "a": 1, }</code>	<code>{ "a": 1 }</code>
Virgola mancante	<code>{ "a": 1 "b": 2 }</code>	<code>{ "a": 1, "b": 2 }</code>
Commenti	<code>{ "a": 1 // nota }</code>	Non sono ammessi
True maiuscolo	<code>{ "ok": True }</code>	<code>{ "ok": true }</code>

VS Code sottolinea in rosso gli errori nei file .json: è il controllo più rapido.

Scrivi e ripara

1 · SCRIVI

In un file **ordine.json** descrivi un ordine con: id, email del cliente, data, un elenco di **due righe** (prodotto, quantità, prezzo) e un campo che dica se è stato pagato.

2 · TROVA I 4 ERRORI

```
{  
  "id": 'o-1',  
  cliente: "mario@example.com",  
  "pagato": False,  
  "totale": 16970,  
}
```

Git in tre comandi

Git tiene la storia dei file di un progetto. Il materiale del corso è su GitHub: per seguirlo ti bastano tre comandi.

```
git clone https://github.com/mg-i-roc-maz/corsi
```

Scarica il repo, **una volta sola**

```
git pull
```

Scarica le novità, **a ogni lezione**

```
git status
```

Mostra che cosa hai modificato

git pull si lancia dentro la cartella **corsi**. Se segnala un conflitto con i tuoi file: **git stash** e poi di nuovo git pull.

Checklist per la lezione 1

Comando	Deve rispondere
<code>git --version</code>	git version 2.x
<code>docker run hello-world</code>	Hello from Docker!
<code>code --version</code>	Un numero di versione di VS Code
<code>ls corsi/microservizi</code>	lezione-1, pre-corso e presentazione-corso.pdf
<code>node -v</code>	v22 o successiva, oppure usa il Dev Container

Qualcosa non va? Segui **lezione-1/start/SETUP.md** o scrivi ai docenti prima della lezione.

QUATTRO DOMANDE PRIMA DI CHIUDERE

1. Che differenza c'è tra un percorso assoluto e uno relativo?

2. Una risposta 404 e una 500: di chi è la colpa?

3. Con quale metodo HTTP crei un nuovo ordine?

4. Perché `{ name: 'a' }` non è JSON valido?

PROSSIMO PASSO

Lezione 1 • Dal monolite ai microservizi

Userai subito tutto: il terminale per avviare i server, HTTP e JSON per parlare con loro.

```
cd corsi/microservizi/lezione-1/start
```